Generative Model. 生成模型.

- Last Time: Self-Supervised Learning 自监督学习.

试图构建出"无标签训练"的范式.

{ pretext task: 自监督学习目标 → 转移表征知识. { 旋转. 拼图. 重建 … ⇒ 几何/像素扰动
下游任务. 对比学习.

对比学习: 将相似的放一起, 不相似的分开.

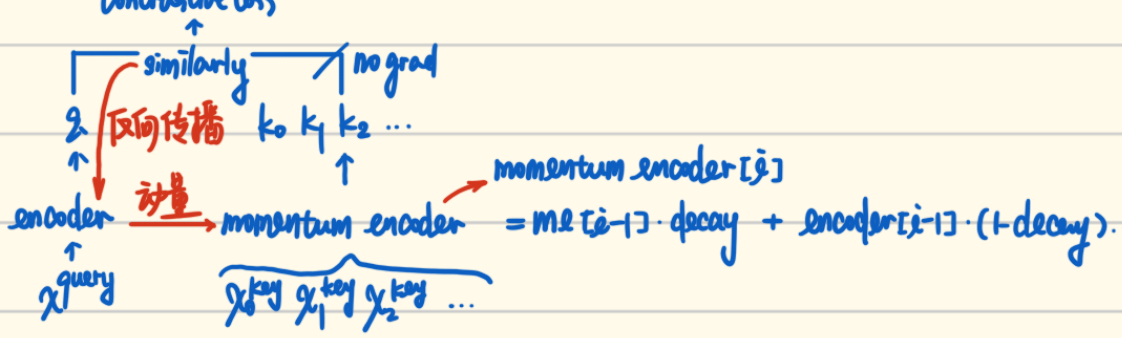
{ 输入图像的扰动集 (一般大小为2). 输入 encoder (CNN. ViT …).
应用"对比"概念. 利用特别的 loss 定义 (InfoNCE loss).

SimCLR Problem: 需要非常大的 batch size.

MoCo: 动量对比方法自监督学习.

核心: 保留一个样本队列 (从之前的训练迭代中).

Contrastive Loss



DINO: KL divergence loss. → DINO v2: 非常强大的自监督特征模型.

(1.42亿图像).

- Today: Generative Models

{ 图像. 文字. 视频生成模型 …
从10年前的完全无成效的领域, 到如今真正有成效.

相比起模型架构. 数学原理. 改变并没有想象的大. 而其背后的算力提升. 稳定的训练方式. 更大数据集. 与分布式计算支撑的扩大规模的能力是推动进步的最大原因.

- Supervised vs Unsupervised Learning

监督学习：

{ Data：(x, y)，(data + label)。
Goal：function map $x \to y$

无监督学习：

{ Data：x (no label)
Goal：Learn hidden structure in data.

聚类，降维分布，拟合概率分布
...

- Generative vs Discriminative Models.

判别模型 "概率性"。

学习概率分布 $P(y|x)$。
(输入 x 条件下标签的分布)。

判别模型的本质即是
让正确的标签在概率分布中尽量竞争到更高的值。

"归一化"：$\int_x p(x)dx = 1$。

"无法拒绝不合理输入"（不属于任何标签）。

生成模型：学习概率分布 $P(x)$。

所有可能出现的图像都竞争概率

{ "合理的"图像种类。→ 结构。
可以分享给不合理的输入图像 → 为其分配一个低，甚至0的概率质量。

用于分享给无效图
学习图像表征（无标签）。

条件生成模型：学习 $p(x|y)$。在输入标签 y 的情形下求 x 概率。

> 生成新的数据，真正关心和实践的方向。

y：一个标签，一段文字，一张图像 + 一段文字。

条件生成模型在一个广泛的空间 x 中得到广泛的输出 x 的概率，是一个非常有难度和不确定的问题。
这需要模型有对语义对象深入推理的能力。

By 贝叶斯规则：

$$p(x|y) = \frac{p(y|x)}{p(y)} p(x)$$

判别　生成　条件生成　标签先验。

原则上，我们可以从两个中获得第三个。

- 生成模型

Modeling ambiguity 建模不确定性：给定输入 y，输出 x 可能有多个 x 可供选择。
模型需给出 P(x|y) 的所有分布。

- 语言模型：输入 文本 y ⟶ 输出 文本 x

Write me a short rhyming poem about generative models ⟶ *They sample from a learned **P**,*
A distribution—structured, free.
Each token comes conditionally,
On all the ones that used to be.

(from GPT).

↑ 可预想的是，事实上符合条件的 x 有很多。

} 输出应对的不唯一性。

- Text to Image：输入 文本 y ⟶ 输出 图像 x
- Image to Video：预测输入图像将发生的视频。

- Taxonomy of Generative Models 生成模型的分类。

- 生成模型是一个具有更多数学运算的子领域

{ model. 建模方式，概率分布…
loss 损失，正确的预期。

- Generative models
  - Explicit density 显式密度，模型直接计算出 p(x)。
    能得到 sample 的 p(x)，但采样更为复杂。
    - Tractable density 可追溯密度。
      真正计算 p(x)。 e.g. Autoregressive 自回归。
    - Approximate density 近似密度
      用某种方式估计 p(x)。 e.g. Variational Autoencoder (VAE) 变分自编码器。
  - Implicit density 隐式密度。不能得到样本的 p(x)，但可以采样
    可以通过底层的实现直接取样
    - Direct. 直接方法，直接从底层抽样
      e.g. GAN Generative Adversarial Network。生成对抗网络。
    - (Indirect). 需通过某种迭代方式抽样
      e.g. Diffusion Models。扩散网络。

- Autogressive Model. 自回归模型

· Maximum Likelihood Estimation 最大似然估计.

-给定显式模型和未知参数，求解参数值.

{ 给定数据集 $x^{(1)}, x^{(2)}, x^{(3)}, \cdots, x^{(N)}$. 网络 f. 有 $p(x) = f(x, w)$.

求解 w^*，使得 $\prod_i p(x^{(i)})$ 的值最大. 并认为此 w^* 是正确的 w.

log 技巧：令 $P = \prod_i p(x^{(i)})$. P 最大等价于 $\log P$ 最大.

$\therefore \log P = \sum_i \log p(x^{(i)})$ 乘→加.

$= \sum_i \log f(x^{(i)}, w)$ → Loss function.

背后原理与思想：

假设其真实概率模型 $p(x) = f_T(x, w_T)$

认为随机抽样过程 $x_1, x_2, \cdots, x_n$ 是 $f_T(x, w_T)$ 独立随机抽样，且是（假设）最大概率发生事件. 并依此计算 w^*.

· 在最大似然估计上，进行更多假设以取得进展：

假设：单个样本 $x^{(i)}$ 可以拆为序列 $x^{(i)} = (x_1^{(i)}, x_2^{(i)}, \cdots, x_T^{(i)})$

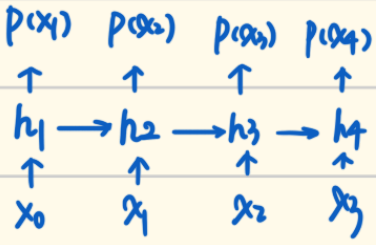
⇓

概率链：$p(x) = p(x_1, x_2, \cdots, x_T)$

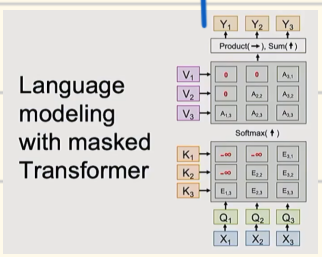
$= p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_2, x_1) \cdots \Longrightarrow$

$= \prod_{t=1}^{T} p(x_t | x_1, \cdots, x_{t-1})$.

① RNNs:



② (masked) transformers



Problem：必须将输入拆分为序列

{ 语言 ∨ 非常适合. 且离散.

图像 ✗ 将图像视为像素序列.（每个像素 [0, 256]）

太贵了. 序列长度非常长.

⇒ 将图像变为非像素的一维序列.

- Variational Autoencoders (VAEs). 自动变分编码器.

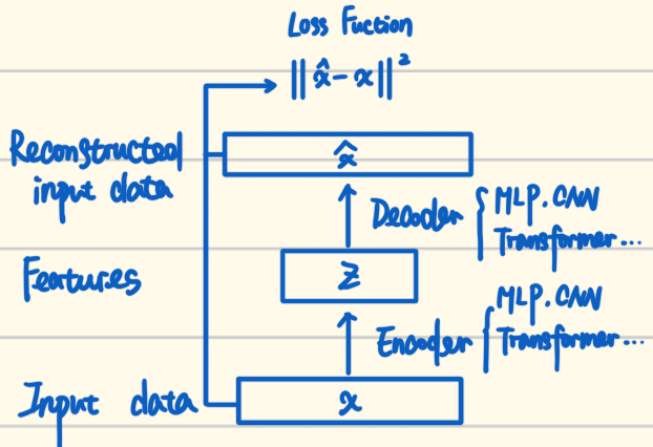
- Autoencoders (AEs). 自动编码器.

Idea：一种无监督学习，从输入 x 中学习 features z. 无标签.

Features 应当包含图像隐含信息并可用于下游编码.

关键：如何进行无标签学习？ → 利用 decoder 重建 input.

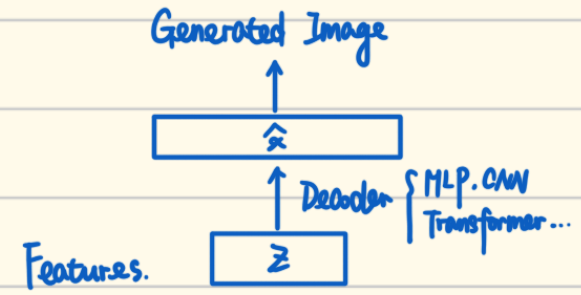
"限制方法"：特征层 z 远小于输入 x. 瓶颈.



P：在完成无监督学习后，怎样利用它来生成？

A：使用Decoder. 在训练中，Decoder得到了从特征 → 图像的能力.

问题转化为 Sample z，通过某个程序采样z.



P：生成一个新的z也许不比生成一个新的x简单.

S：如果可以在无监督训练过程中迫使z服从已知的分布（如高斯分布等），就能合理地采样.

- 迫使特征z在其空间内保持某种概率分布结构，就是VAEs想完成的目标.

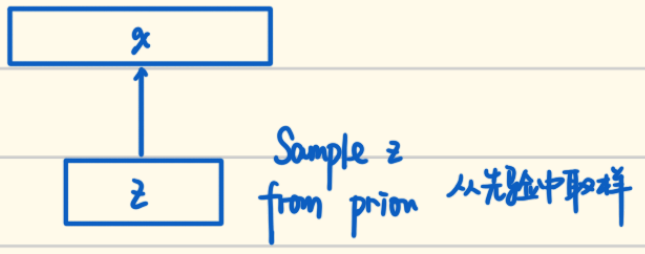
- Variational Autoencoders（VAEs）.

在 autoencoders 上的概率论证：

假设训练数据 $\{x^{(i)}\}_{i=1}^{N}$，是由潜在向量表征z生成的.

$x^{(i)}$ ← decoder. $z^{(i)}$

并假设简单先验，例如z服从高斯分布



train：基本思路：最大似然

① $p_\theta(x) = \int p_\theta(x,z)dz = \int p_\theta(x|z)\,p_\theta(z)dz$.　Problem：z是超高维向量，这个积分无法计算甚至难以近似.

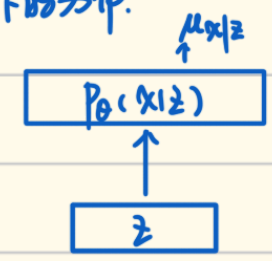
decoder项 假设高斯分布.

② $p_\theta(x) = \dfrac{p_\theta(x|z)p_\theta(z)}{p_\theta(z|x)}$　decoder 高斯分布.　Problem：无法计算 $p_\theta(z|x)$

Solution：使用另一个神经网络学习 $q_\phi(z|x) = p_\theta(z|x)$.

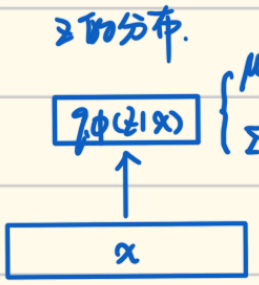
Decoder 网络：

输入z，输出x在z条件下的分布.



Encoder 网络：

输入x，输出潜在变量z的分布.



分开计算，并强制假设 $p_\theta(x|z) = N(\mu_{x|z}, \sigma^2)$，$q_\phi(x|z) = N(\mu_{z|x}, \Sigma_{z|x})$

对decoder，σ^2是设定的超参，decoder学习 $\mu_{x|z}$.

对encoder，输出均值向量和协方差向量.

train goal:

$$p_\theta(x) = \frac{p_\theta(x|z)\,p(z)}{p_\theta(z|x)}$$

$$\log p_\theta(x) = \log \frac{p_\theta(x|z)\cdot p(z)}{p_\theta(z|x)} = \log \frac{p_\theta(x|z)\,p(z)\,q_\phi(z|x)}{p_\theta(z|x)\,q_\phi(z|x)}$$

最大化 $\log p_\theta(x)$

$$= \log p_\theta(x|z) - \log \frac{q_\phi(z|x)}{p(z)} + \log \frac{q_\phi(z|x)}{p_\theta(z|x)}$$

D_{KL}：衡量变量差异.

$$= E_z[\log p_\theta(x|z)] - E_z[\log \frac{q_\phi(z|x)}{p(z)}] + E_z[\log \frac{q_\phi(z|x)}{p_\theta(z|x)}].$$

$$= E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z)) + D_{KL}(q_\phi(z|x), p_\theta(z|x))$$

数据重建：对 $q_\phi(z|x)$ 给出的分布，求 $\log p_\theta(x|z)$ 的期望

Prior：Encoder 的输出应当匹配 z 的高斯分布. 此项计算模型输出与先验的匹配程度.

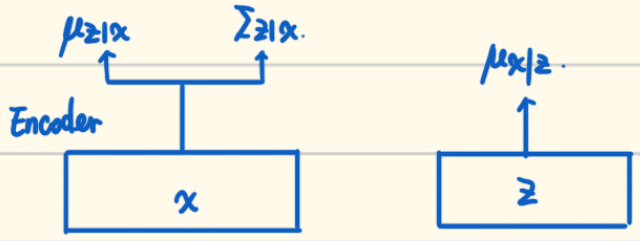
我们正是利用 $q_\phi(z|x)$ 来"代替" $p_\theta(z|x)$（因为无法计算 $p_\theta(z|x)$.）所以令此项为0以得到**最终下限**

$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z))$$ 作为优化目标.

$\log p_\theta(x)$ 实质上不依赖 z，

所以 $E_z(\log p_\theta(x)) = \log p_\theta(x)$

- Training: $E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z))$

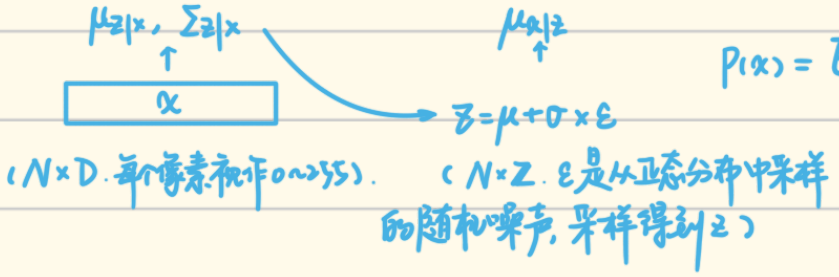


1. 在encoder上运行 x，得到 $\mu_{z|x}$，$\Sigma_{z|x}$.
2. $q_\phi(z|x) = N(\mu_{z|x}, \Sigma_{z|x})$.
3. 从 $q_\phi(z|x)$ 抽样得到 z.
4. 在decoder上运行，得到 $\mu_{x|z}$.
5. 计算 $E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)]$.

Sampling：在prior上取样 z，运行decoder，得到参数 $\mu_{x|z}$ 后对高斯分布采样，生成图像

注释：一些关于过程自己的理解：

① 整个模型：



（$N\times D$，每个像素视作0~255）

（$N\times Z$，ε 是从正态分布中采样的随机噪声，采样得到 z）

(ELBO)

$$p(x) = E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x), p(z))$$

② 先验与最大似然原则：评估模型好坏（loss）⟶ 最大似然：$\prod p(x^{(i)})$ / $\sum \log p(x^{(i)})$ 有最大值

⟶ 近似，ELBo（evidence lower bound.）

Lec 14. 生成模型(二).

今日学习：Implicit density.

模型对概率进行隐式建模，这使我们不能计算$p(x)$.

上一课中的模型由于能计算$p(x)$，所以采用先验 + 最大似然($p(x)$)计算loss.

- Generative Adversarial Network (GANs). 生成对抗网络.

放弃建模$p(x)$，但允许从底部分布中采样.

- Setup：数据集$\{x_i\}$. 理论上的x_i来源的分布$p_{data}(x)$. 目标是从p_{data}中采样.

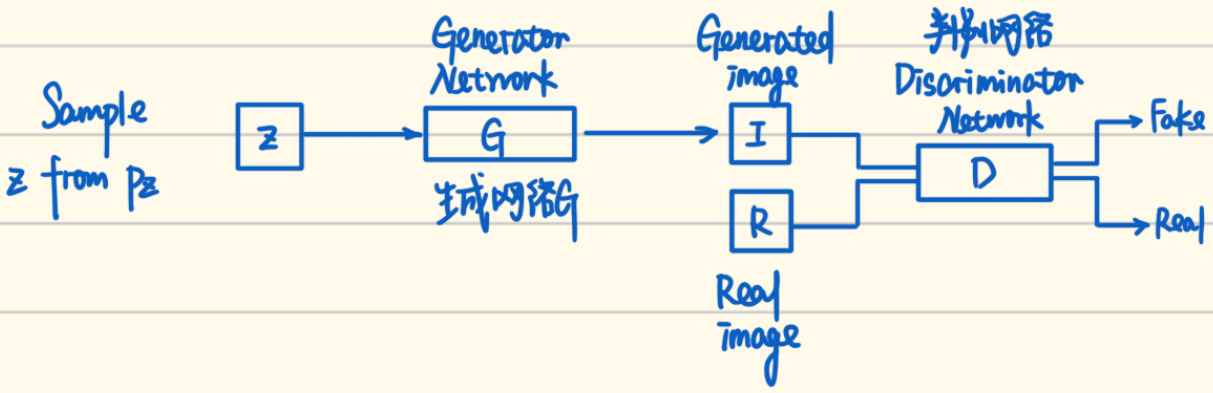
- Idea：引入潜在变量z.（拥有简单先验形式，例如高斯）.

从$p(z)$中采样z，通过网络G，得到$x=G(z)$.

此时z与网络G共同构成p_G，可视作一种分布，x是它的样本。目标是p_G与p_{data}相匹配.

Problem：应该怎么衡量p_G与p_{data}，或者其样本$x=G(z)$与$\{x_i\}$的相似程度呢？

Solution：引入判别器网络.



同时训练D.G.

希望最后能使p_G接近p_{data}

直觉：判别网络和生成网络在对抗中. D分辨真实图像的能力增强，G生成也会越来越趋于真实.

- Train（理论）. 通过"极大极小博弈"联合训练D.G：

$$\min_G \max_D \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}[\log(1-D(G(z)))] \right)$$

$D(x)$：0 分辨器认为Fake；1 分辨器认为Real

实际将给出0~1中间值.

对D：$\max_D \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}[\log(1-D(G(z)))] \right)$

分辨p_{data}内容时，$D(x)$尽量为1

分辨p_G内容时，$D(x)$尽量为0.

对G则相反：

$$\min_G \left(E_{x\sim p_{data}}[\log D(x)] + E_{z\sim p(z)}[\log(1-D(G(z)))] \right)$$

与G无关. … D(x)尽量为1.

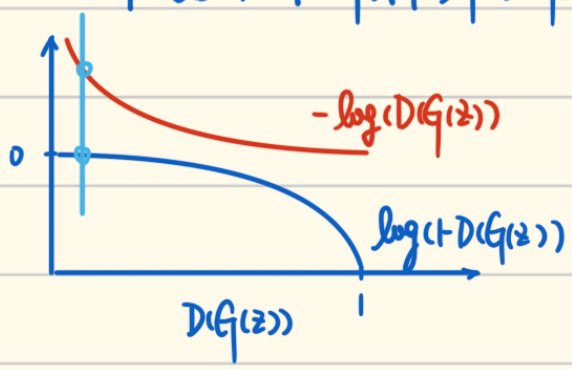
→ $\min_G \max_D V(G,D)$. → 优化过程：$\begin{cases} D = D + \alpha_D \frac{dV}{dD} & \text{最大化} \\ G = G - \alpha_G \frac{dV}{dG} & \text{最小化.} \end{cases}$

↑ V不是损失函数，

对整个系统来说取决是G.D的角力.

这意味着我们难以评估系统的训练进度.

Problem：在训练初期，D(G(z))非常小（非常易被判别），梯度趋于0.



Solution：对生成器，将其log改为 $-\log(D(G(z)))$，从而在开始时得到高梯度.

可数学上计算得，DG的收敛值为 $\begin{cases} D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x)+p_G(x)} \\ p_G(x) = p_{data}(x). \end{cases}$

$\begin{cases} D,G\text{假设为无限取样.} \\ D,G\text{被假设为无穷能力（实际网络能力有限）.} \end{cases}$

- GAN Architectures：

DC-GAN

G,D通常是CNN. DC-GAN是首个可被使用的生成对抗网络.

其第一作者 Alec Radford 也是GPT-1和GPT-2的第一作者.

StyleGAN.

更复杂的架构，更好的实践结果.

- GANs：隐式空间插值 Latent Space Interpolation

GANs的隐性空间具有平滑的特征：

例如，若有z_0, z_1两个隐向量，并在它们中插值

$\begin{cases} z_t = tz_0 + (1-t)z_1 \\ x_t = G(z_t). \end{cases}$ 得到的图像x_t也会"平滑"地在两个样本中变化.

- Summary:

Pros:

- 非常简单的公式. 方程
- 可以得到非常好的结果.

Cons:

- 没有loss可以观察和调整
- 训练非稳定(易爆炸.)
- 乘以扩大数据.

生成对抗网络是~2016-2021最常用的生成网络.

- Diffusion Models 扩散模型.

- 警告:该领域术语.符号体系非常混乱.在许多种数学形式化方法.论文间在术语.符号上差异巨大.
- 仅在本节涉及现代的"干净"实现(Rectified Flow).

- Intuition. 直觉.

单位高斯分布 $z \sim N$. 与图像等大.

噪声水平 $t \in (0,1)$. 数据 x. 得到不同噪声水平的样本 x_t.

- 训练神经网络清除噪声 $f_\theta(x_t, t)$.
- 推理:先绘制随机样本 $x_1 \sim p_{noise}$. 在 x_1 上反复应用 f_θ.

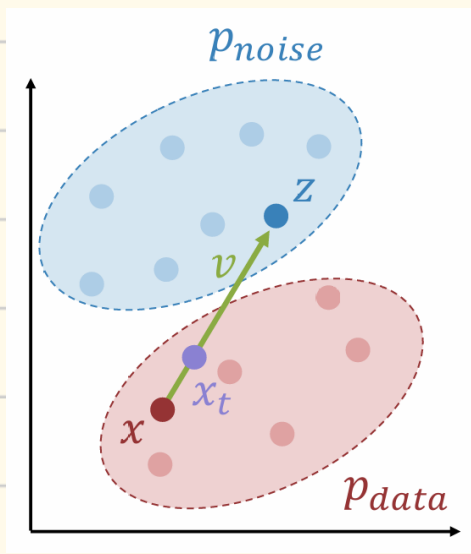
(上述是一个高水平的概述.实际上的术语解释与实现过程多有变体).

- Rectified Flow Models. 整流.校正流模型.

假设分布 p_{noise}(通常友好.如单位高斯等).

p_{data}(宇宙中对图像的分布.高维.难以概述).

> 保有同样的难度.



- train iteration:

取样 $z \sim p_{noise}$ $x \sim p_{data}$ $t \sim Uniform[0,1]$.(噪声水平).

令 $\vec{v} = \vec{z} - \vec{x}$. $x_t = (1-t)x + tz$. (在整流模型,t意味着线性插值.)

训练 f_θ. $L = \|f_\theta(x_t, t) - v\|_2^2$.

Core training loop is just a few lines of code!

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Rectified Flow Model 的核心仅仅几行代码!

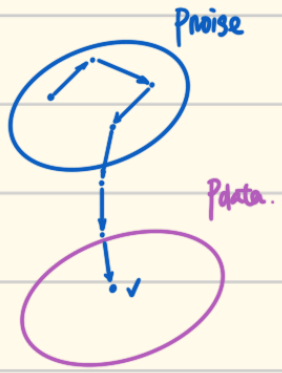
- Sampling:

1. 选择 steps T (often T=50).
2. Sample $x_1 \sim p_{noise}$
3. For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:

$$v_t = f_\theta(x_t, t)$$

$$x = x - v_t/T$$

Return x.



Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```

- 条件整流 Conditional Rectified Flow.

{ 有条件生成模型很容易适应整流模型.
  将条件作为额外的输入信号给入模型.

Problem: 能否控制模型对输入 y 的专注度?

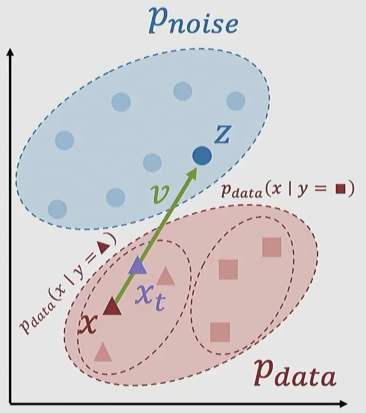
Conditional Rectified Flow

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Model", 2022

- Classifier-Free Guidance (CFG) ★

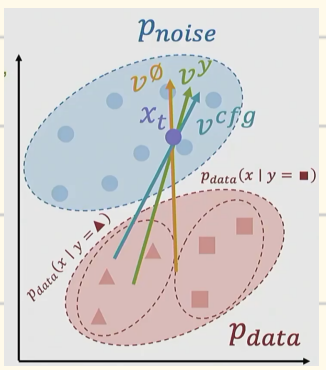
在训练时掷硬币，随机插入空值.

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```



模型在插入空值时会被迫学习"无条件"路径.

此时模型就同时有两条路径. 可以依此调节方向.

⇓

采样时同时计算 $\begin{cases} v^\emptyset = f_\theta(x_t, y_\emptyset, t) \\ v^y = f_\theta(x_t, y, t) \end{cases}$

令 $v^{cfg} = (1+w)v^y - wv^\emptyset = v^y + w(v^y - v^\emptyset)$.

↓ 预测时使用 v^{cfg} 步进.

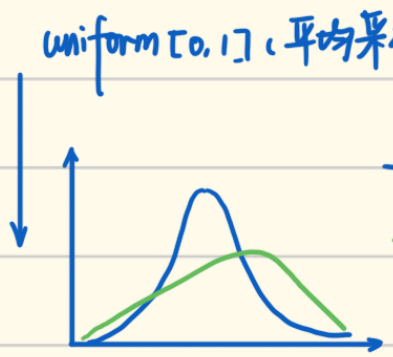
w: 增强幅度.

- Noise Schedules.

最早的模型平均得到 t. 但事实上：

- t=1. 全噪音： 模型几乎没有训练意义. 简单指向数据平均值.
- t=0. 无噪音： 最优预测. 指向噪声平均值.

当 t 位于中间位置时：可能有多对 noise - data 相匹配. 模型需要分辨出最佳方向



#高分辨率数据：
- 相邻像素具有更强的关联性.
- 噪声需要更强的水平以销毁这些关联性.
  - shift to higher noise. 更高的噪声强度.

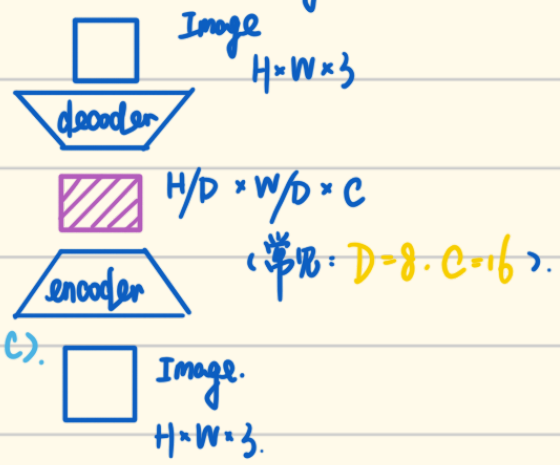
然而扩散模型在面对高分辨率图像时并不能 work naively.

⇓

- Latent Diffusion Models (LDMs). 实际上当代最流行的模型架构.

-多阶段过程.

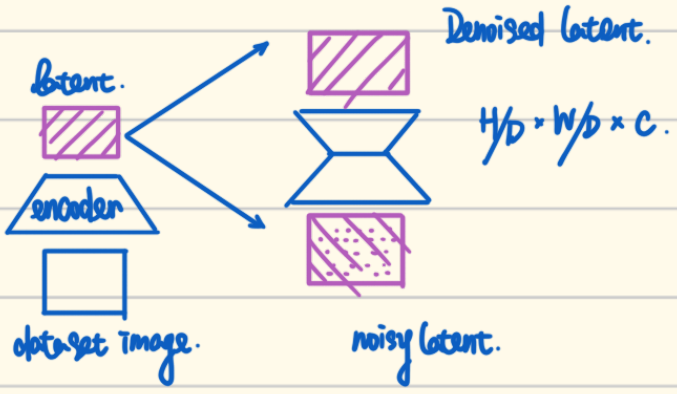
① 训练 encoder - decoder 网络，将 image 转为 latents. (潜在空间).



encoder / decoder 通常是带注意力的 CNNs.

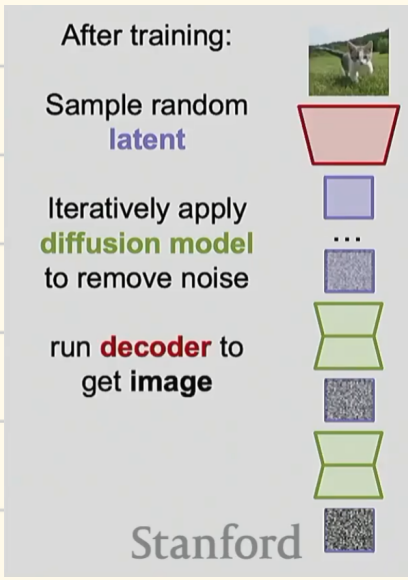
将 image (H×W×3) 转化为 latent ($\frac{H}{D}, \frac{W}{D}, C$).
减少 H.W. 增加 C.

② 在 Latents 层上训练扩散模型.



- 利用 stage ① 训练得来的 encoder，将图像转换为 latent，再以此训练 diffusion model.
- freeze encoder. 仅调整扩散模型.

③ After training：（推理过程）.



- 采样latent噪声并在扩散模型上运行多次，以得到"干净"的latent.
- 得到latent后利用stage①中的decoder，将latent转换为图像

- Stage①：encoder + decoder ： VAE！

引入VAE：① 结构相似，都是需要中间的采样层.

② 单纯的AE（不加入对中间层先验的限制）会导致latent流形的形状不优美，diffusion model在上面不能得到好的结果.

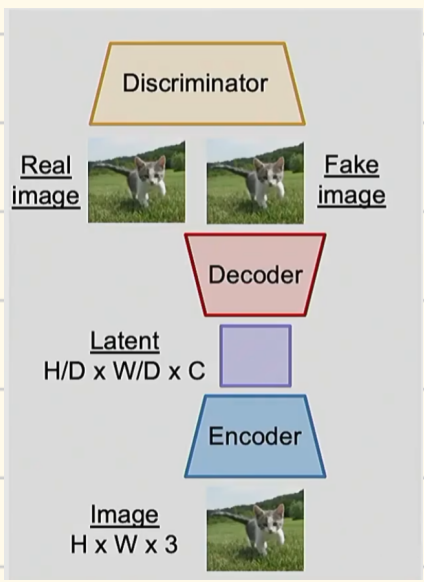
③ VAE采用的loss中包含一个KL项，可以让latent更倾向于高斯这种分布，扩散模型能得到更好的结果，同时扩散模型能够得到包含"输入"的生成，也补足了VAE的缺陷.

（CVAE得到的条件生成有一个有趣现象：

- 将encoder/decoder拼一个条件向量"进行训练，结果发现条件越精确，输出图像越模糊.
- VAE的KL散度条件要求特征空间的平滑性，但这与条件的多样性产生冲突
- VAE的loss本质是$\|x-\mu\|^2$，这使模型对拿不准的情形，更倾向于寻找一个通用的中间解：不论输入猫、狗、熊，它都输出一个平均的、三个都像又都不像的解.）

Problem：VAE的引入会让decoder得到的图像模糊.

Solution：引入一个discriminator并重构loss.（GAN结构）.



LDM：VAE + GAN + diffusion model.

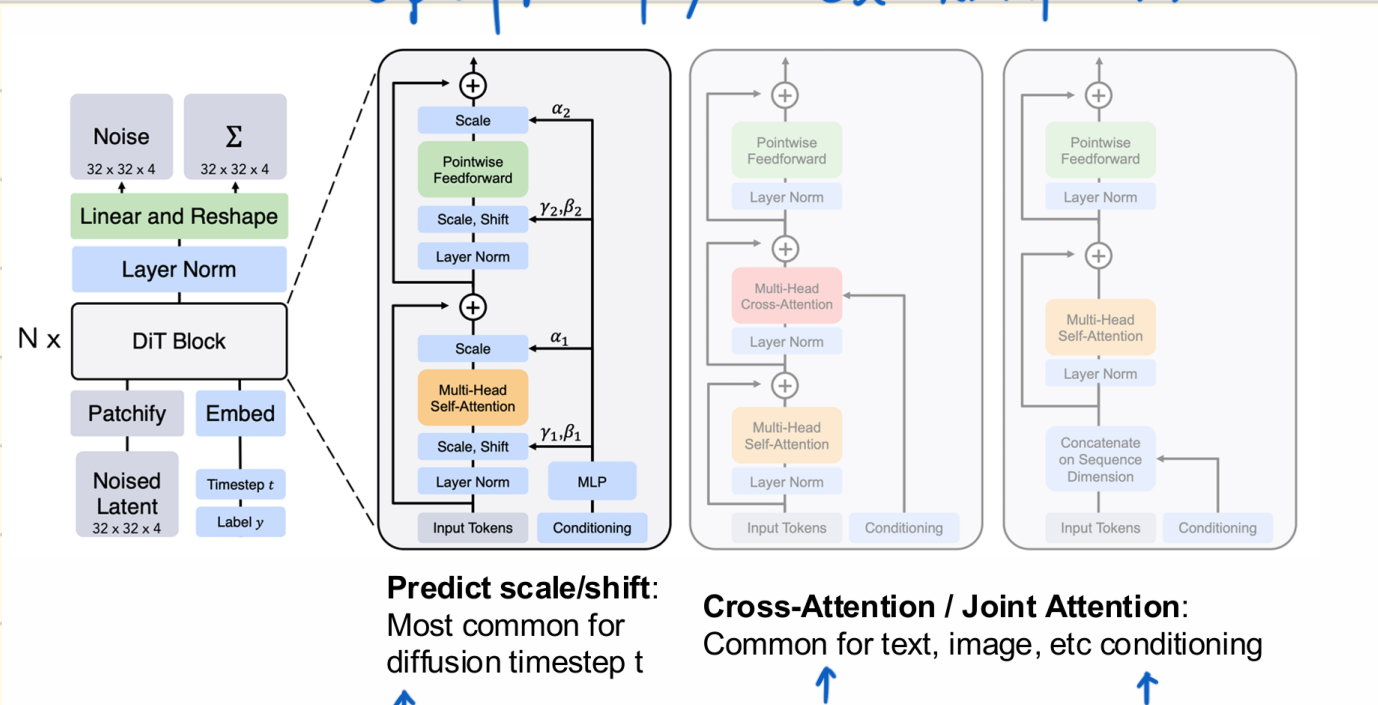
- Diffusion Transformer (DiT).

扩散模型使用transformer块来进行架构.

回顾外层黑盒: model(x_t, t, y).
- x_t: 带噪声的图像输入
- t: 噪声强度. 也可理解为步长./时间戳(对应推理时的循环).
- y: 条件输入.

- 对于transformer blocks, 怎样注入 t, y ?

① 预测缩放/平移. ② 交叉注意力/联合注意力.



Predict scale/shift: Most common for diffusion timestep t

Cross-Attention / Joint Attention: Common for text, image, etc conditioning

在Block中插入缩放. 移动更以注入信息. 常用于t的注入(信息少).

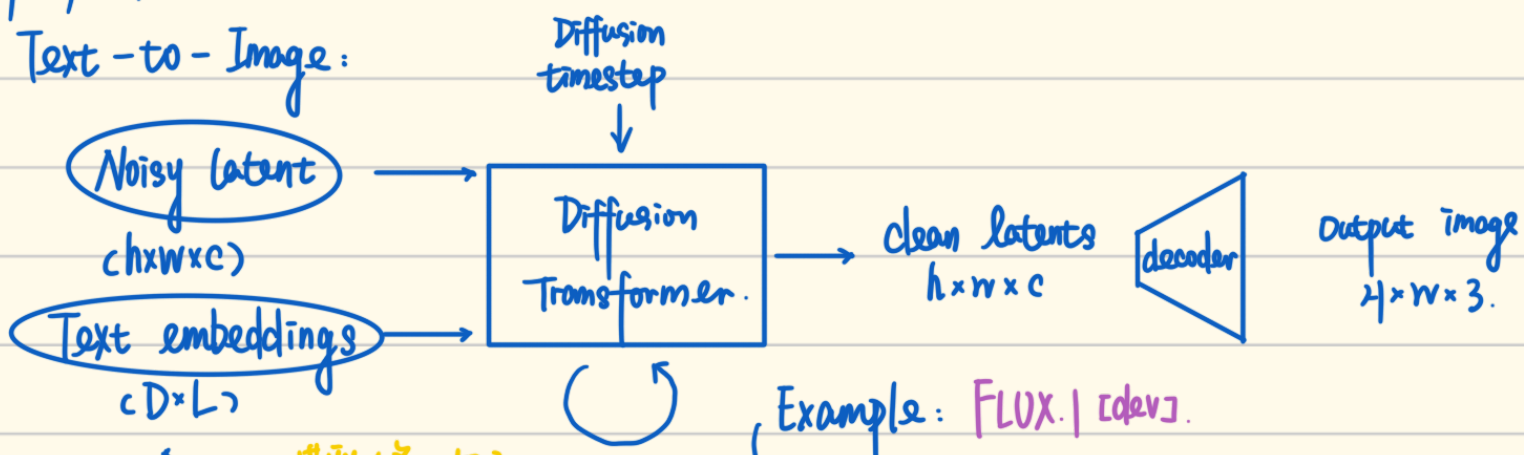
向block中插入一个 多头cross-attention层. condition作为K.V的输入

将condition插入到序列 的新维度中. 结构强

对y的注入. 包括text. image等.

- LDM的应用:

- Text-to-Image:



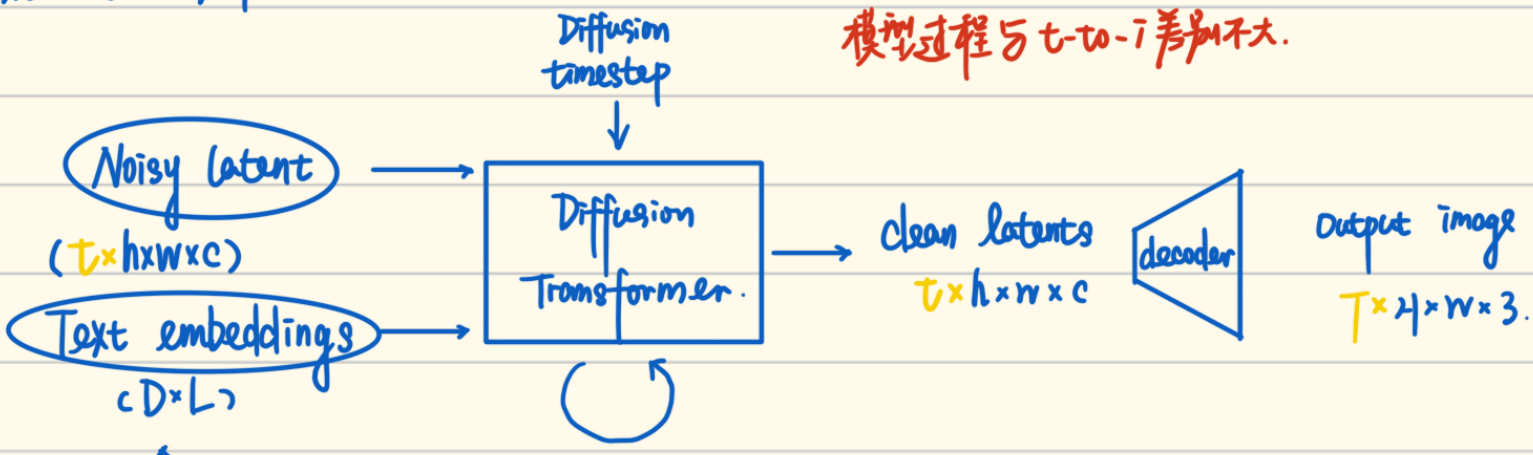
pretrained text encoder (T5, CLIP等).

模型太多训不了. 用预训练的.

text →

Example: FLUX.1 [dev].
- Text Encoder: T5+CLIP
- Encoder/Decoder: 8×8 下采样. 128×128×16 ⟷ 1024×1024×3.
- Diffusion model: 12B参数量. 2×2 patch. (64×64. 即1024 tokens).

- Text - to - Video



模型过程与t-to-i差别不大.

Example: Meta MovieGen

Text Encoder: UL2. ByT5. MetaCLIP

Encoder/Decoder: 8×8×8 downsample

Diffusion model: 30B参数量. 1×2×2 patch → 76k tokens.

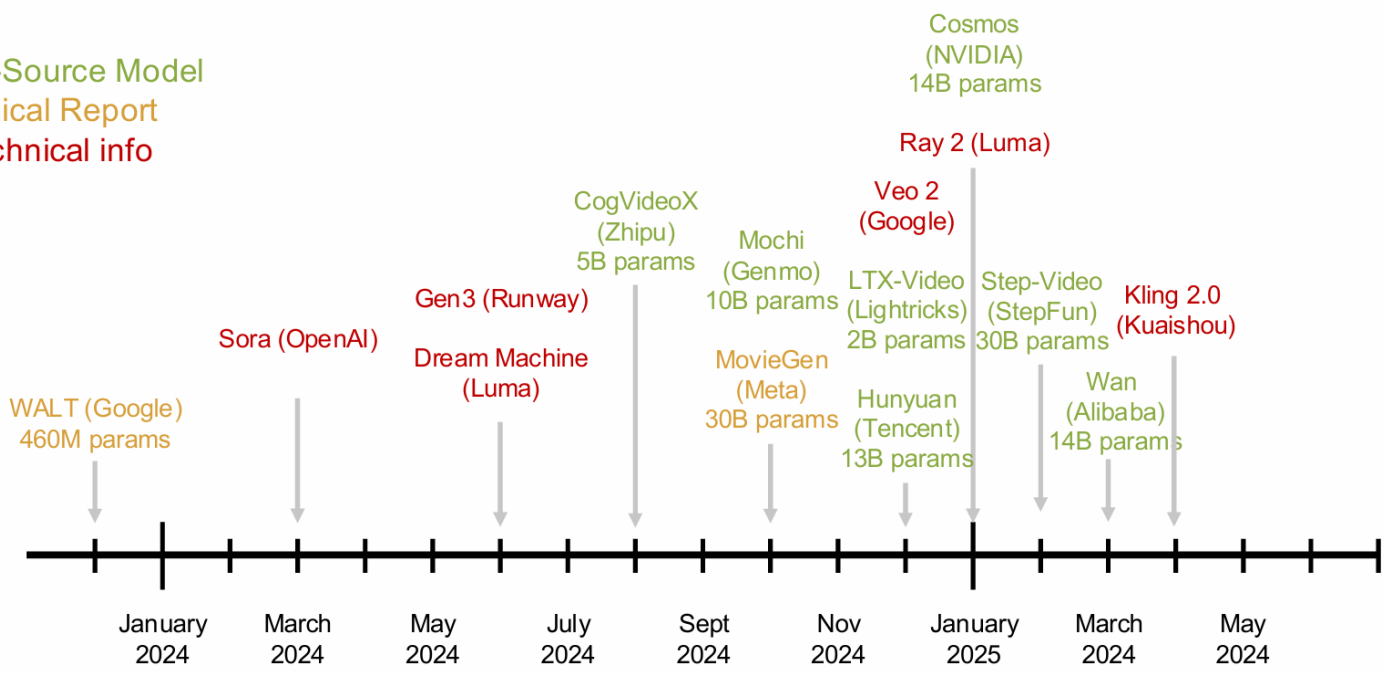
32×128×72×16 → 33×128×72×16

253×1024×576×3

花费相当惊人.

The Era of Video Diffusion Models

Open-Source Model
Technical Report
No technical info



- Diffusion Distillation

diffusion model 的一大劣势在于: 采样期间需要多周期迭代. really slow!

⇓

Solution: distillation 蒸馏. (略过核心算法原理不讲.)

- 以某种方式修改模型, 牺牲部分质量以大幅减少迭代步数(甚至一步).

- 当前活跃的研究领域.

- Generalized Diffusion 广义扩散.

Rectified Flow:

1. Sample $x \sim p_{data}$. $z \sim p_{noise}$

   $t \sim p_t$

2. Set $x_t = (1-t)x + z$.

   Set $y_{gt} = z - x$.

3. Compute $y_{pred} = f_\theta(x_t, t)$

   Compute loss:

   $\|y_{gt} - y_{pred}\|_2^2$

Generalized Diffusion:

1. Sample $x \sim p_{data}$. $z \sim p_{noise}$

   $t \sim p_t$

2. Set $x_t = a(t)\,x + b(t)\,z$

   Set $y_{gt} = c(t)\,x + d(t)\,z$.

3. Compute $y_{pred} = f_\theta(x_t, t)$

   Compute loss:

   $\|y_{gt} - y_{pred}\|_2^2$

核心差异：如何计算 x_t, y_{gt}？

x_t. y_{gt} 通常是 x. z 的线性组合，且系数与 t 相关.

⇓

RF: $\begin{cases} a(t) = 1-t \\ b(t) = t \\ c(t) = -1 \\ d(t) = 1. \end{cases}$

another flavor:

VP (variance preserving): $\begin{cases} a(t) = \sqrt{\alpha(t)} \\ b(t) = \sqrt{1-\alpha(t)}. \end{cases}$

若 x. z 相互独立且具有单位方差，则输出的混合也有单位方差.

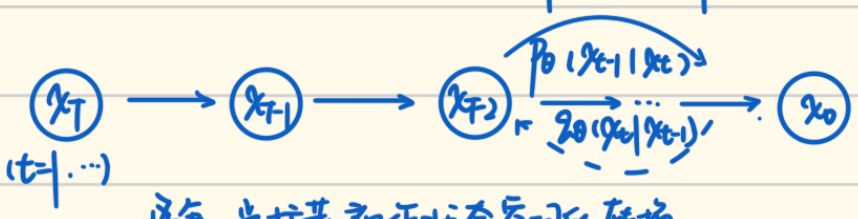
VE (variance Exploding): $\begin{cases} a(t) = 1 \\ b(t) = \sigma(t). \end{cases}$

....

How to choose these functions?

- 通常由数学引导：（分析数学观点）.

- Diffusion is a Latent Variable Model 潜在变量模型.



将每一步扩散视作状态空间的转换.

利用 VAE 的思想. 优化 lower bound.

- Score Function.

$s(x) = \frac{\partial}{\partial x} \log p(x)$. $p(x)$ 的导数构成一个向量场. 使其指向密度更高（概率更大）的区域.

扩散学习一个近似于 $s(x)$ 的函数.

- Stochastic Differential Equations. 随机微分方程.

↓

Perspectives on Diffusion

Great blog post by Sander Dieleman:

https://sander.ai/2023/07/20/perspectives.html

(All his blog posts are great)

1. *Diffusion models are autoencoders*
2. *Diffusion models are deep latent variable models*
3. *Diffusion models predict the score function*
4. *Diffusion models solve reverse SDEs*
5. *Diffusion models are flow-based models*
6. *Diffusion models are recurrent neural networks*
7. *Diffusion models are autoregressive models*
8. *Diffusion models estimate expectations*

#开坑. 从作业笔记 assignment 3 得到的对去噪扩散模型(DDPM)的新理解:

- Forward Process 前向过程:

令数据集对应的图片分布为 $q(x_0)$. 定义一个前向增噪过程(马尔可夫链). 其中每一步为:

$$q(x_t|x_{t-1}) \sim N(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I).$$

均值 $\sqrt{1-\beta_t}\, x_{t-1}$. 方差 $\beta_t I$.

- $(\beta_1, \beta_2, \dots, \beta_T)$ 是分步方差, 决定噪声分布规律.

由高斯分布的性质, 整个过程 $q(x_t|x_0)$ 可表示为:

$$q(x_t|x_0) \sim N(x_t; \sqrt{\bar{\alpha}_t}\, x_0, (1-\bar{\alpha}_t) I).$$

均值累乘, $\bar{\alpha}_t \to 0$.

$$\begin{cases} \alpha_t = 1-\beta_t \\ \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_t \end{cases}$$

也就是说, 若 $(\beta_1, \beta_2, \dots, \beta_T)$ 设置得当, 在多步后可得到 $q(x_T)$ 趋近于纯高斯噪声 $N(0, I)$.

$$q(x_T) = \int q(x_T|x_0) \cdot q(x_0)\, dx_0 \quad \text{与 } x_0 \text{ 无关.}$$

$$\to \int N(0, I)\, q(x_0)\, dx_0 = N(0, I) \int q(x_0)\, dx_0 = N(0, I).$$

顺带, 对 $x \sim N(\mu, \sigma^2)$ 可使用 $\sigma\varepsilon + \mu$ 完成. ε: 标准高斯分布采样.

故可轻易完成 $q(x_t|x_{t-1})$ 和 $q(x_t|x_0)$ 的采样. ($x_t \overset{\text{采样}}{=} \sqrt{1-\bar{\alpha}_t}\,\varepsilon + \sqrt{\bar{\alpha}_t}\, x_0$).

一个关于这里概率写法的小问题: 这里 $q(x_t|x_0)$ 应该是 $q(X_t|X_0 = x_0)$. 代表 X_t 在已知 x_0 的分布规律.

这个 q 也不太懂是何意味. 就先按正常概率论里的 p 理解了, 表示概率密度函数

- Reverse Process 恢复过程.

令 $p(x_{t-1}|x_t)$ 是 $q(x_t|x_{t-1})$ 的反向过程. (即已知 x_t 下的 x_{t-1} 的分布)

两个关键洞察:

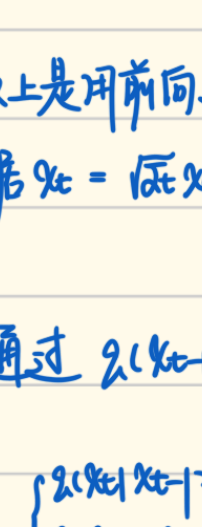
① $p(x_{t-1}|x_t)$ 具有任意复杂的形态, 几乎不可能是高斯分布. 这让建模、采样变得困难.

② 若前向过程的分步噪声 β_t 足够小, 则后向过程 $p(x_{t-1}|x_t)$ 接近高斯分布.

上述两点提示我们，若将恢复过程拆成一步一步的恢复，就可使用高斯分布方便建模采样。
（并且要确保步数足够多以使 p_t 足够小！）。

所以说现在的目标是用模型拟合 $p_\theta(x_{t-1}|x_t)$ 代替 $p(x_{t-1}|x_t)$，并且依此采样得到 x_{t-1}。
同时由于 $p(x_{t-1}|x_t)$ 可近似视作高斯分布，则 p_θ 可以直接视作 $N(x_{t-1}; \mu_\theta(x_t,t), \Sigma_\theta(x_t,t))$
（x_{t-1}：表示此式描述 x_{t-1}；μ：均值；Σ：协方差）。

也就是希望模型（含参数 θ）：



在 Denoising Diffusion Probabilistic Models.（Ho. 等）中，经过平均数学推导，
直接采用 $\Sigma_\theta = \beta_t I \,/\, \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t I$（$\bar\alpha_t = \prod_{s=1}^{t}\alpha_s$）。
（$\beta_t I$：单位矩阵；$\frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t I$：数学推论；近似，效果差不多）

但在更现代的模型中（如 Improved DDPM），它是通过学习得到的。
（在 Σ_θ 的两个值中间插值）。

一 降噪与最优化目标：

（最大似然，让分布采样得到数据集样本概率最大化）。

最小化数据集样本的期望负对数 $E(-\log p_\theta(x_0))$　　$p_\theta(x_0) = p(x_T) \times \prod_{t=1}^{T} p(x_{t-1}|x_t)$。

然而与 VAE 一样，扩散模型本算不出概率，采用另外的手段：

去噪损失（loss）：$E_{t,x_0,\epsilon}[\|\epsilon - \epsilon_\theta(\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\epsilon, t)\|^2]$

> Ho. 等人证明，目标等价。

- ϵ_θ：神经网络函数，接受带噪图和 t（时间步），输出噪声 ϵ。
- $\sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\epsilon$：带噪图，$= x_t$。
- t：时间步，在 $1 \sim T$ 间均匀分布。
- ϵ：噪声采样。

（给一张图 x_0，用 ϵ 采样出带噪图 x_t）。

但是求噪声 ϵ_θ 和求 μ_θ 有何关系？

本质上是用前向过程增噪并采样 $x_t \sim q(x_t|x_0)$，传入网络后求出 ϵ_θ 并与噪声 ϵ 计算 loss。

根据 $x_t = \sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon$，可以倒求出 x_0。（式略）

即 $q(x_{t-1}|x_t) = q(x_{t-1}|x_t, x_0)$

再通过 $q(x_{t-1}|x_t,x_0) = \frac{q(x_t|x_{t-1},x_0)\,q(x_{t-1}|x_0)}{q(x_t|x_0)} = \frac{q(x_t|x_{t-1})\,q(x_{t-1}|x_0)}{q(x_t|x_0)}$

（马尔可夫过程，x_0 在此式可去掉）　　此三式都可以求出！（都是高斯分布）

$$\begin{cases} q(x_t|x_{t-1}) \sim N(x_t; \sqrt{1-\beta_t}\,x_{t-1}, \beta_t I) \\ q(x_{t-1}|x_0) \sim N(x_{t-1}; \sqrt{\bar\alpha_{t-1}}x_0, (1-\bar\alpha_{t-1})I) \\ q(x_t|x_0) \sim N(x_t; \sqrt{\bar\alpha_t}x_0, (1-\bar\alpha_t)I) \end{cases} \xrightarrow{\text{详见论文}} \begin{cases} q(x_{t-1}|x_t,x_0) = N(x_{t-1}; \tilde\mu_t(x_t,x_0), \tilde\beta_t I) \\ \tilde\mu_t(x_t,x_0) = \frac{\sqrt{\bar\alpha_{t-1}}\beta_t}{1-\bar\alpha_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}x_t \\ \tilde\beta_t = \frac{1-\bar\alpha_{t-1}}{1-\bar\alpha_t}\beta_t \quad (\text{与 } x_0 \text{ 无关的常数}) \end{cases}$$

又因为前文，利用噪声 ϵ 可求出 $\hat{x}_0$，我们将其代入即可得到 $p(x_{t-1}|x_t) = q(x_{t-1}|x_t, \hat{x}_0)$

#为什么预测 ϵ？

1. ϵ 是单位高斯分布，训练稳定。（预测 x_0 会在不同 t 时产生不稳定波动）
2. ϵ 的 loss 在数学上与 $\nabla_{x_t} \log p(x_t)$ 等价。
3. 预测 x_0 效果非常差。

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 采样一个 x_0
3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 随机取 t
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 随机取 ϵ（噪声）
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged 根据 loss 进行学习。

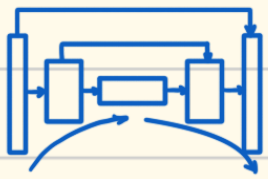
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 将 x_T 取为随机噪声
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 添加随机噪声
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 计算 x_{t-1}。
5: **end for**
6: **return** $\mathbf{x}_0$

参考：

https://github.com/datawhalechina/deep-learning-notes/tree/main/zh/ch14-diffusion-models

U-Net：专用于图像→图像任务的神经网络架构。



编码器：（下采样）

- 将输入图像转化为空间分辨率↓，特征维度↑的分层特征

解码器（上采样）：

- 逐步恢复空间分辨率以采样特征。
- 结构与编码器一致。

每层解码器会将对应编码器层的特征也拼入输入。

直接通道，捕捉细节。

减轻瓶颈层负担，使其专注高水平表示而非记忆细节。